

Metodología de Sistemas I

Unidad N°1: “Herramientas de Repositorios de Software y Colaboración”

- 1. Cloud Computing (Computación en la Nube).
- 2. Arquitectura de Microservicios.
- 3. IA Aplicada al Desarrollo (Ingeniería de Software Aumentada).

AGENDA

1. Cloud Computing (Computación en la Nube)

- A. Modelos de Servicio (La "Pila" de la Nube).
- B. Modelos de Despliegue.
- C. Características Técnicas Esenciales (NIST).
- D. Impacto en la Ingeniería de Software.

A. Modelos de Servicio (La "Pila" de la Nube).

- ✓ IaaS (Infraestructura como Servicio): El proveedor ofrece el hardware virtualizado (servidores, redes, almacenamiento). El ingeniero es responsable de instalar el Sistema Operativo, las bases de datos y la aplicación. Ejemplo: Amazon EC2, Google Compute Engine.
- ✓ PaaS (Plataforma como Servicio): El proveedor ofrece el entorno de ejecución (runtime), el sistema operativo y las bases de datos. El ingeniero solo se preocupa por el código de la aplicación. Ejemplo: Heroku, AWS Lambda, Google App Engine.
- ✓ SaaS (Software como Servicio): El software ya está construido y listo para el usuario final. El ingeniero aquí actúa más en la integración que en la construcción base. Ejemplo: Microsoft 365, Salesforce.

B. Modelos de Despliegue.

- ✓ Nube Pública: Recursos compartidos en internet (escalabilidad masiva).
- ✓ Nube Privada: Infraestructura dedicada exclusivamente a una organización (máxima seguridad y control).
- ✓ Nube Híbrida: Una combinación donde se mantienen datos sensibles en privado y se escala a lo público para picos de demanda.

C. Características Técnicas Esenciales (NIST).

- ✓ Autoservicio bajo demanda: El desarrollador puede obtener recursos (más CPU, más RAM) automáticamente sin intervención humana del proveedor.
- ✓ Acceso amplio a la red: Disponible desde cualquier dispositivo y ubicación.
- ✓ Reserva de recursos (Pooling): Los recursos físicos se reparten entre múltiples usuarios de forma invisible.
- ✓ Elasticidad rápida: Capacidad de crecer (scale-up) o encogerse (scale-down) en segundos según el tráfico.
- ✓ Servicio medido: Se paga exactamente por lo que se consume (como la luz o el agua).

D. Impacto en la Ingeniería de Software.

- ✓ Diseño para el Fallo: En la nube, se asume que un servidor puede morir en cualquier momento. El software debe ser capaz de reiniciarse en otro nodo sin perder datos.
- ✓ Escalabilidad Horizontal: En lugar de hacer un servidor más grande, el software debe ser capaz de funcionar en 10, 100 o 1000 servidores pequeños simultáneamente.
- ✓ DevOps y Automatización: Como la infraestructura es "código", la creación de servidores y redes se automatiza mediante scripts, eliminando errores manuales.

2. Arquitectura de Microservicios

- A. Definición y Filosofía.
- B. Características Técnicas Fundamentales.
- C. Componentes de Soporte (El Ecosistema).
- D. Ventajas y Desafíos (La Realidad Técnica).

A. Definición y Filosofía.

Es un enfoque para desarrollar una aplicación única como un conjunto de pequeños servicios, cada uno ejecutándose en su propio proceso y comunicándose con mecanismos ligeros, generalmente una API de recursos HTTP (REST) o mensajería (RabbitMQ, Kafka).

B. Características Técnicas Fundamentales.

1. Descentralización de Datos: Cada microservicio tiene su propia base de datos. Esto evita que un cambio en el esquema de una tabla afecte a todo el sistema, aunque introduce el desafío de la consistencia eventual.
2. Gobierno Políglota: Permite utilizar el lenguaje de programación o la base de datos más adecuada para cada servicio específico. Un servicio de IA puede estar en Python, mientras que el motor de transacciones está en Java.
3. Despliegue Independiente: Se puede actualizar o corregir un solo servicio sin necesidad de redistribuir o reiniciar toda la aplicación. Esto reduce drásticamente el "tiempo de salida al mercado" (Time-to-Market).
4. Aislamiento de Fallos: Si el servicio de "Recomendaciones" falla, el usuario aún puede "Comprar" y "Buscar" productos. El sistema se degrada con elegancia en lugar de colapsar por completo.

C. Componentes de Soporte (El Ecosistema).

- API Gateway: Un punto de entrada único que redirige las peticiones de los clientes al microservicio correspondiente.
- Service Discovery: Un directorio dinámico donde los servicios se registran para que otros puedan encontrarlos automáticamente en la red.
- Circuit Breaker (Disyuntor): Un patrón de diseño que evita que un servicio siga llamando a otro que está fallando, protegiendo la estabilidad de la red.

D. Ventajas y Desafíos (La Realidad Técnica).

Aspecto	Ventaja	Desafío (Complejidad)
Escalabilidad	Se escalan solo los servicios con alta demanda.	Mayor latencia por comunicación en red entre servicios.
Desarrollo	Equipos pequeños y autónomos por servicio.	Dificultad para realizar pruebas de integración extremo a extremo.
Mantenimiento	Código más pequeño y fácil de entender.	Complejidad operativa (requiere monitoreo avanzado y trazabilidad).

3. IA Aplicada al Desarrollo (Ingeniería de Software Aumentada)

- A. Generación y Asistencia de Código (Coding Assistants).
- B. Pruebas y Aseguramiento de Calidad (AI-Augmented Testing).
- C. Refactorización y Deuda Técnica.
- D. Gestión de Proyectos y Estimación.
- E. Desafíos Éticos y Técnicos (La Responsabilidad del Ingeniero).

A. Generación y Asistencia de Código (Coding Assistants).

- ✓ Completado Autogestivo: La IA sugiere líneas de código o bloques funcionales completos (funciones, clases) basándose en el contexto del archivo actual.
- ✓ Traducción de Lenguajes: Capacidad de convertir lógica de un lenguaje a otro (ej: de COBOL a Java) con una precisión alta, facilitando la modernización de sistemas legados.
- ✓ Explicación de Código: Permite que un ingeniero reciba una explicación en lenguaje natural de un bloque de código complejo o mal documentado.

B. Pruebas y Aseguramiento de Calidad (AI-Augmented Testing).

- ✓ Generación de Casos de Prueba: Algoritmos que analizan el flujo del código y generan automáticamente unit tests y pruebas de integración para cubrir todos los caminos lógicos.
- ✓ Self-healing Tests (Pruebas Auto-sanables): Si la interfaz de usuario cambia levemente (un ID de un botón cambia), la IA ajusta los scripts de prueba automáticamente sin que el ingeniero tenga que intervenir manualmente.
- ✓ Detección de Vulnerabilidades: Escaneo proactivo para encontrar fallos de seguridad (como inyecciones SQL o desbordamientos de búfer) antes de que el código llegue a producción.

C. Refactorización y Deuda Técnica.

- ✓ Identificación de "Code Smells": La IA detecta patrones de diseño mediocres o código redundante y propone refactorizaciones alineadas con los principios de Clean Code.
- ✓ Optimización de Rendimiento: Sugerencia de algoritmos más eficientes para reducir el consumo de memoria o CPU, impactando directamente en la eficiencia del software.

D. Gestión de Proyectos y Estimación.

- ✓ Predicción de Plazos: Analiza el historial del equipo y la complejidad de las tareas para ofrecer fechas de entrega más realistas que las estimaciones humanas manuales.
- ✓ Análisis de Riesgos: Identifica qué módulos del sistema tienen mayor probabilidad de introducir errores basados en su historial de cambios y complejidad.

E. Desafíos Éticos y Técnicos (La Responsabilidad del Ingeniero).

- ✓ Alucinaciones: La IA puede generar código que parece correcto pero es lógicamente erróneo o inexistente (llamadas a librerías que no existen).
- ✓ Propiedad Intelectual: Dilemas legales sobre quién es el dueño del código generado por IA y si este infringe licencias de código abierto.
- ✓ Seguridad del Código Generado: El riesgo de que la IA sugiera patrones de código que, aunque funcionales, contienen vulnerabilidades conocidas.

